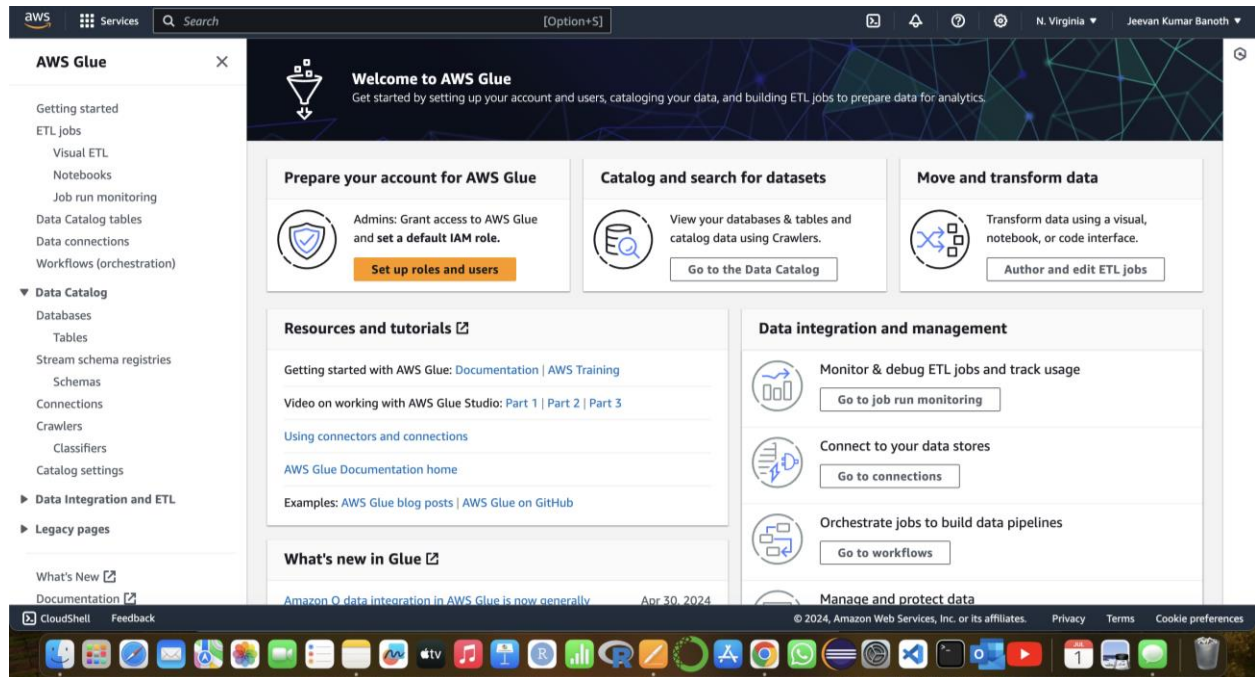


Jeevan Kumar Banoth - 02105145
CIS 602-7103: Special Topics in CIS

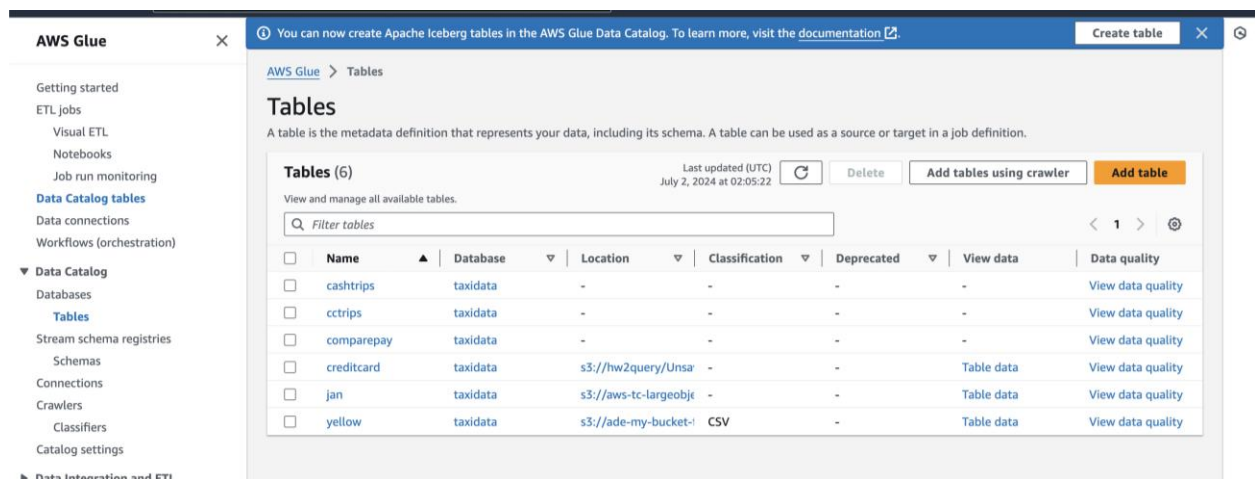
Homework – 3:

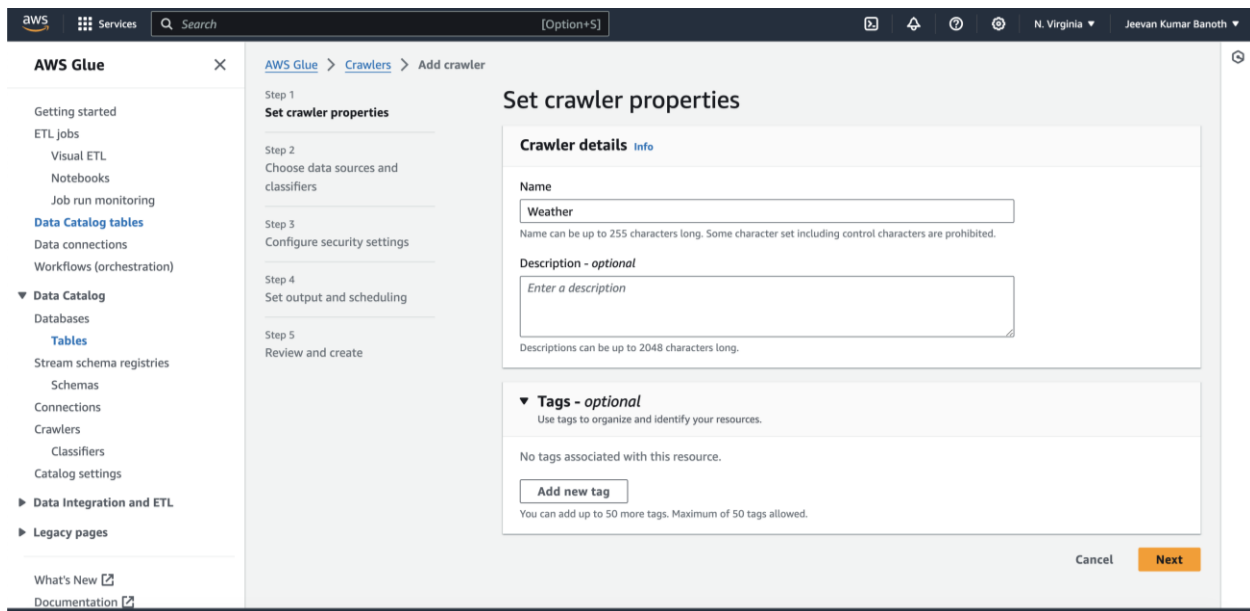
- 1. Configure and create the AWS Glue Crawler: -**

By using the provided lab manual, open the AWS console and search for AWS glue and open it as below.

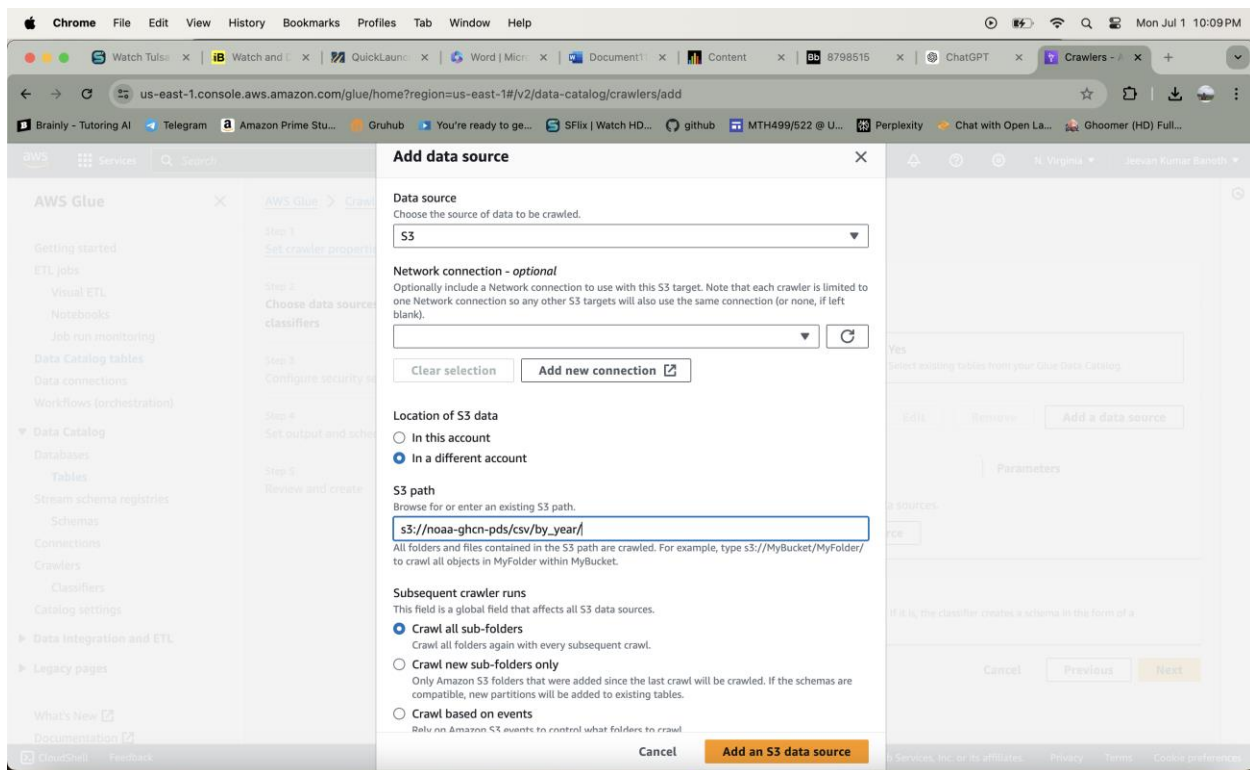


Next by using the tables, we will be creating the crawlers for database. So, follow the steps from the manual for creating crawler.

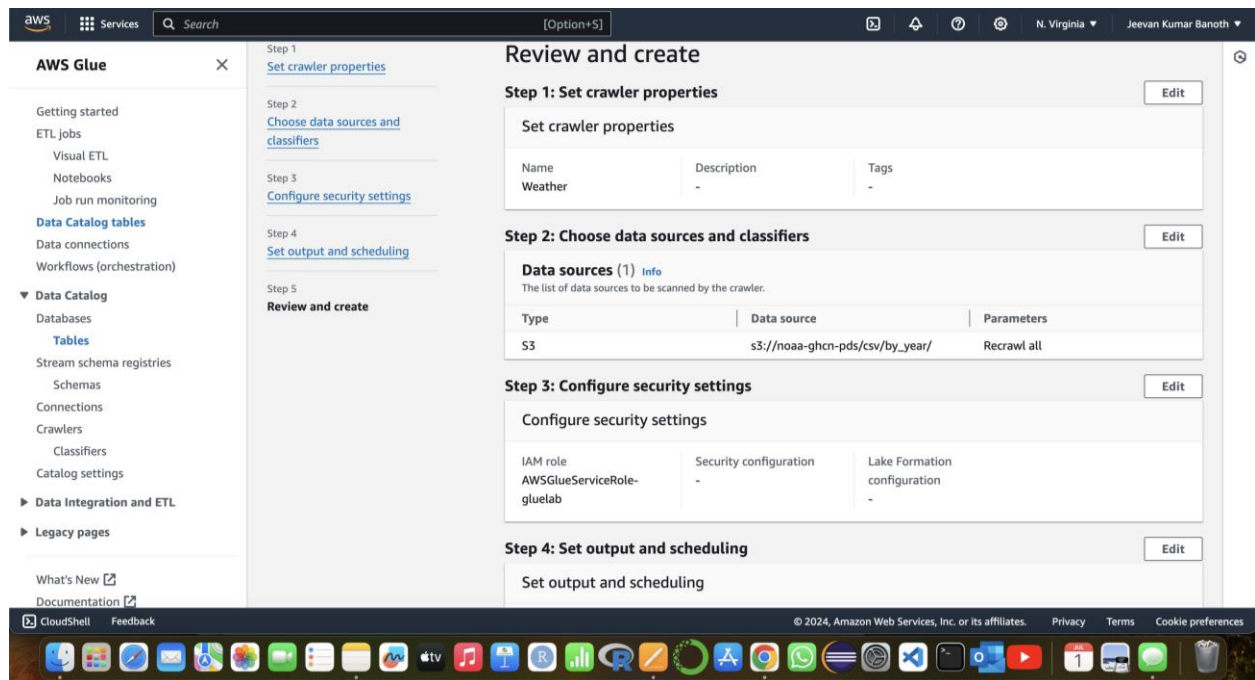




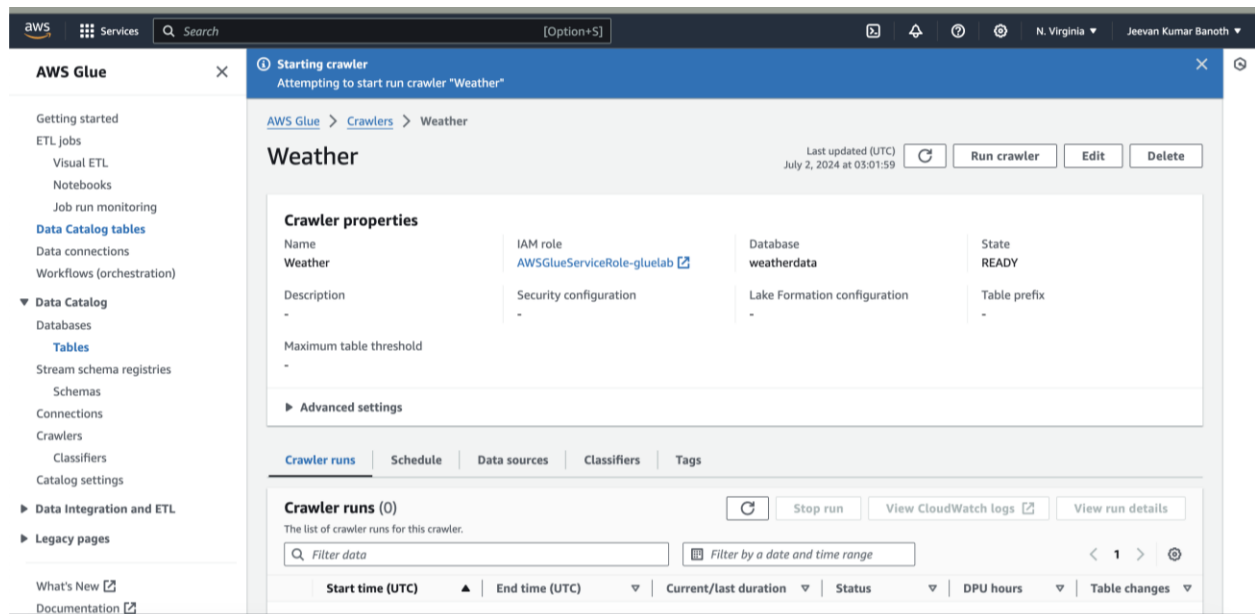
Here, we will be giving the name and tags as per given in the manual, but that is not necessary to do.



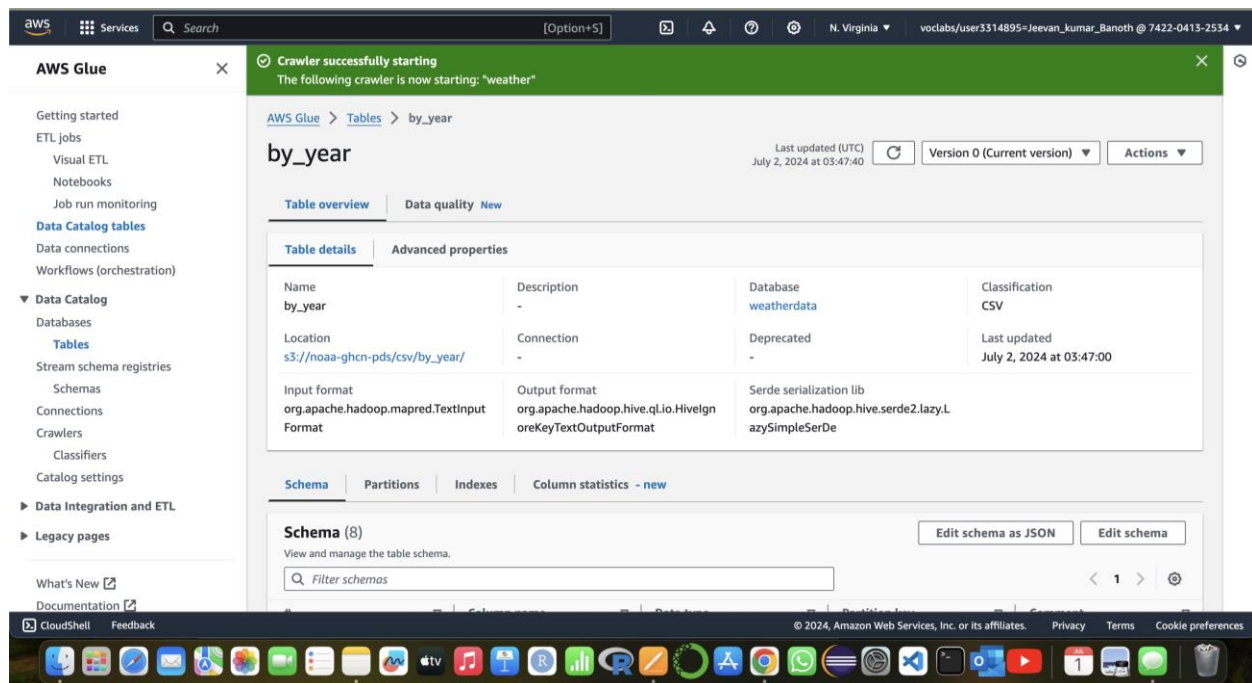
As you can see here, in the next step of adding crawler, we will need to add the data source and S3 path, so I will be performing that as instructed.



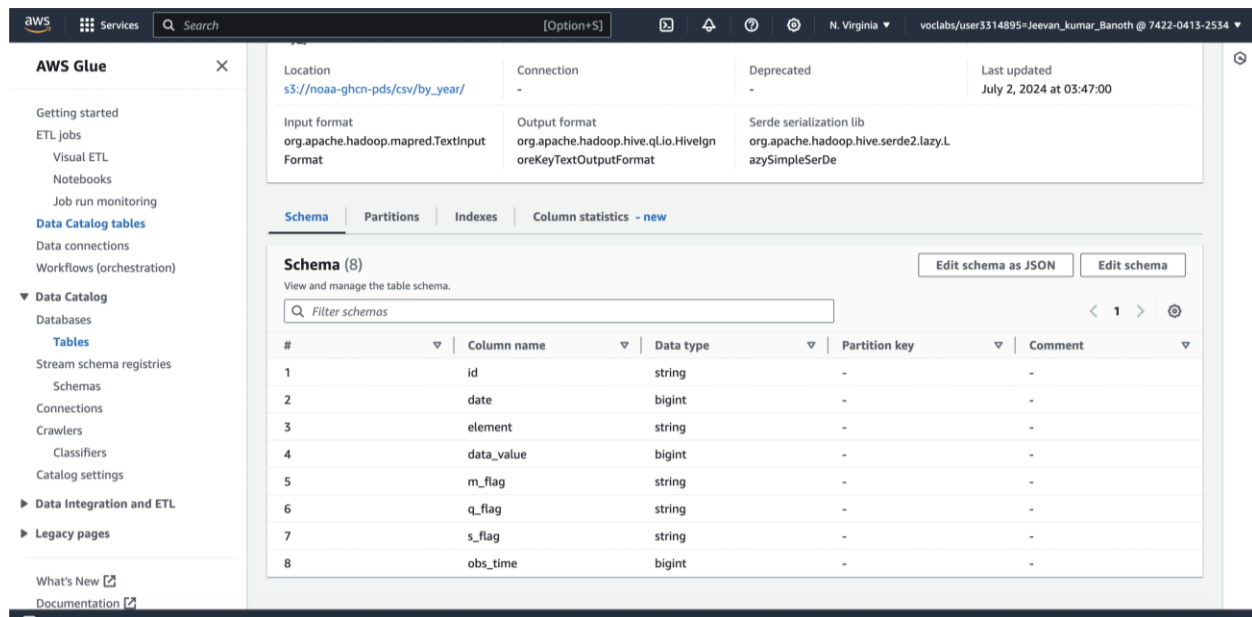
This is going to be the last step in creating a crawler which is review. Here we can also edit if we made any mistakes earlier.



As you can see above, the weather crawler is created successfully and then I will be starting it to check the table by-year in the tables section and to see if it is successfully run or not.

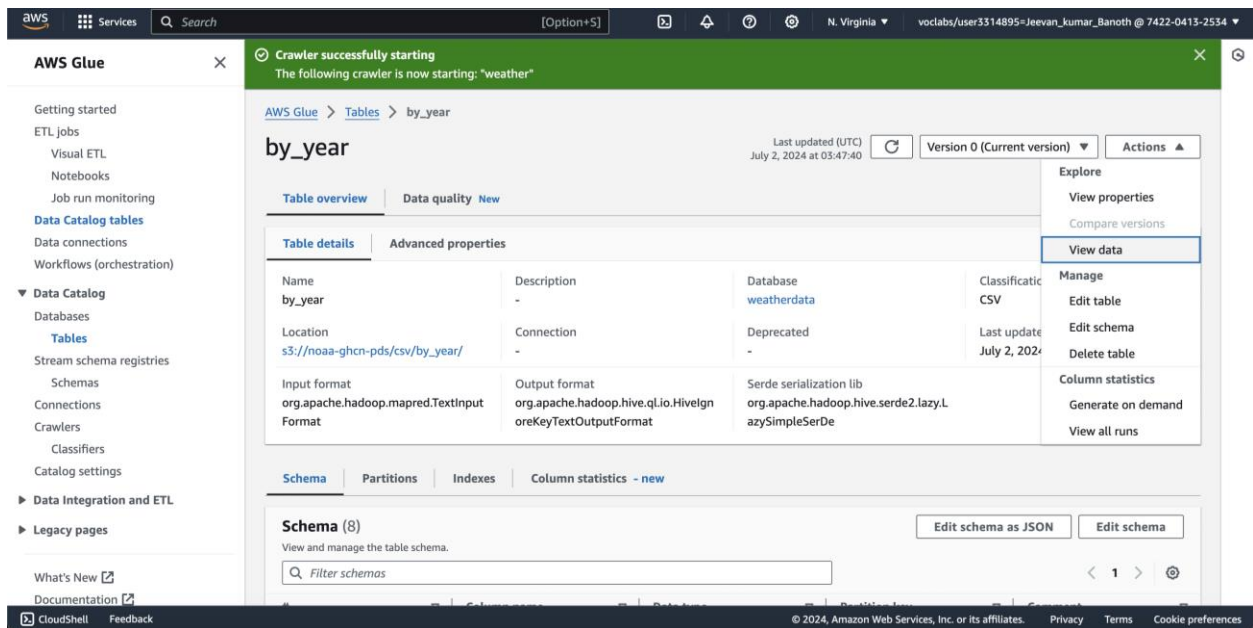


The above image shows the by_year table which is created from the running of crawler.



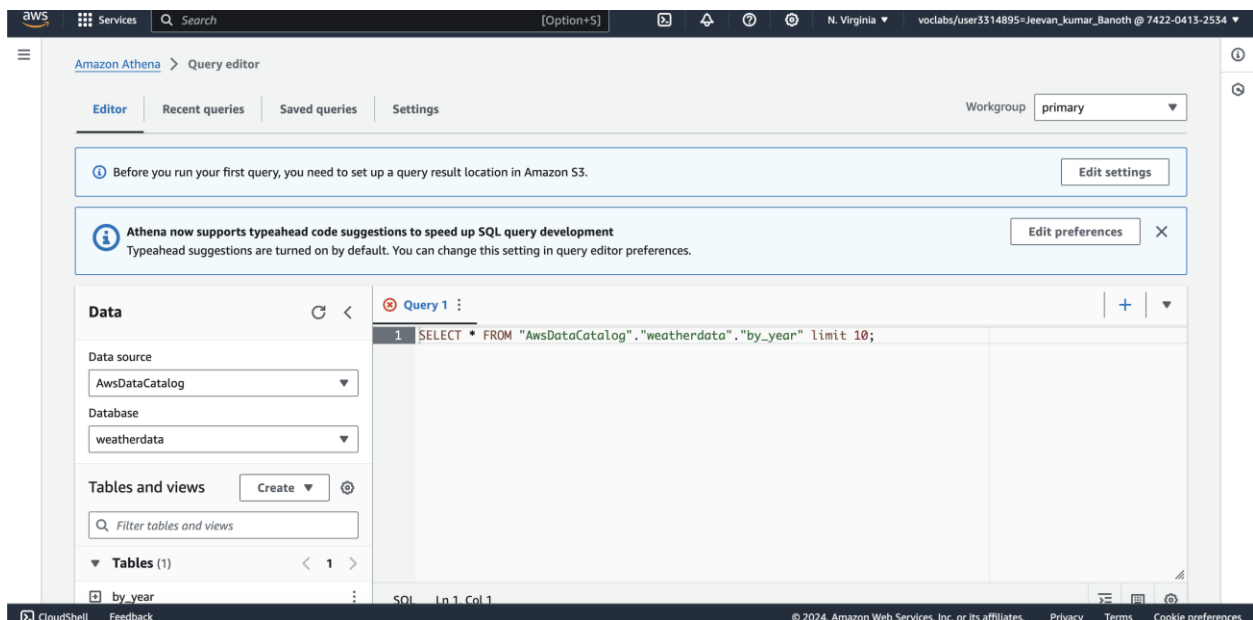
And the above image shows the schema from the by_year table which contains all the keywords of the database. Later, by using this schema, we will write the SQL code for running.

2. Querying a Table by Using Athena



In this task, you will configure an S3 bucket to store Athena query results, preview a database table in Athena, create a table for data after 1950, and run queries on the selected data.

From the actions button > view data then we will be moving to the different console as below which is Amazon Athena.



Now, we will need to do some modifications like selecting bucket and some permissions etc.

The screenshot shows the AWS Glue console interface. On the left, the 'Data' sidebar lists the data source as 'AwsDataCatalog', the database as 'weatherdata', and a table named 'by_year'. The main editor area displays a SQL query: `SELECT * FROM "AwsDataCatalog"."weatherdata"."by_year" limit 10;`. Below the query editor, the 'Run again' button is highlighted in orange. The 'Query results' tab is active, showing a green 'Completed' status bar with the following metrics: Time in queue: 78 ms, Run time: 1.008 sec, and Data scanned: 74.46 KB. The bottom of the console shows the AWS logo, 'CloudShell', 'Feedback', and copyright information for 2024.

As you can see above, errors are cleared, and we got the output for the code. The code has run successfully in 78ms time and 74.46KB of data has been scanned.

The screenshot shows the 'Query results' tab in the AWS Glue console. It displays a table with 10 rows of data. The table has columns: #, id, date, element, data_value, m_flag, q_flag, s_flag, and obs_time. The data is as follows:

#	id	date	element	data_value	m_flag	q_flag	s_flag	obs_time
1	ITE00100554	17720101	TMAX	51			E	
2	ITE00100554	17720101	TMIN	31			E	
3	ITE00100554	17720102	TMAX	48			E	
4	ITE00100554	17720102	TMIN	18			E	
5	ITE00100554	17720103	TMAX	38			E	
6	ITE00100554	17720103	TMIN	18			E	
7	ITE00100554	17720104	TMAX	23			E	
8	ITE00100554	17720104	TMIN	-1			E	
9	ITE00100554	17720105	TMAX	6			E	
10	ITE00100554	17720105	TMIN	-18			E	

The bottom of the console shows the AWS logo, 'CloudShell', 'Feedback', and copyright information for 2024.

The above Image shows the output which is created in the late20th table by using by_year table.

The screenshot shows the AWS CloudShell interface. On the left, the 'Data' pane shows the 'Data source' as 'AwsDataCatalog' and the 'Database' as 'weatherdata'. The 'Tables and views' pane shows two tables: 'by_year' and 'late20th'. The main pane displays the SQL query for 'Query 2':

```
1 CREATE TABLE weatherdata.late20th
2 WITH (
3     format = 'PARQUET',
4     external_location = 's3://labhw/lab3'
5 ) AS
6 SELECT date, element AS type, data_value AS observation
7 FROM by_year
8 WHERE date / 10000 BETWEEN 1950 AND 2015;
```

The query execution status is 'Completed'. The 'Query stats' section shows: Time in queue: 92 ms, Run time: 58.704 sec, Data scanned: 101.53 GB. The message 'Query successful.' is displayed at the bottom.

As you can see in the Image that query has run successfully and time taken was 92ms and data scanned is 101.53GB which is huge and Run time of the code is 58.704 seconds.

The screenshot shows the 'Query results' pane. The query status is 'Completed'. The 'Results (10)' section shows a table with 10 rows. The table has columns: #, date, type, and observation. The data is as follows:

#	date	type	observation
1	20150101	TAVG	4
2	20150101	TMAX	89
3	20150101	TMIN	11
4	20150101	TAVG	45
5	20150101	TMAX	67
6	20150101	TMIN	-72
7	20150101	TAVG	-17
8	20150101	TMAX	56
9	20150101	TMIN	-72
10	20150101	TAVG	-21

This is the output of the late20th table which has been generated by the code that we used just now.

The screenshot shows the AWS CloudShell interface. On the left, the 'Data' section is expanded, showing 'Tables and views'. Under 'Tables (2)', there are 'by_year' and 'late20th'. Under 'Views (1)', there is 'tmax'. The main area displays the SQL query: `1 CREATE VIEW TMAX AS`
`2 SELECT date, observation, type`
`3 FROM late20th`
`4 WHERE type = 'TMAX';`
`5`. Below the query, the 'Query results' tab shows 'Completed' with a status bar indicating 'Time in queue: 83 ms', 'Run time: 544 ms', and 'Data scanned: -'. The message 'Query successful.' is displayed at the bottom.

This code will generate a table called tmax and this is compiled in 544ms (about half second), and we can see the message as Query successful.

The screenshot shows the AWS CloudShell interface with the 'tmax' table selected. The 'Query results' tab shows 'Completed' with a status bar indicating 'Time in queue: 68 ms', 'Run time: 1.947 sec', and 'Data scanned: 40.80 MB'. Below this, the 'Results (10)' section displays a table with 10 rows of data. The table has columns: #, date, observation, and type. The data is as follows:

#	date	observation	type
1	20070101	261	TMAX
2	20070101	256	TMAX
3	20070101	272	TMAX
4	20070101	300	TMAX
5	20070101	267	TMAX
6	20070101	278	TMAX
7	20070101	261	TMAX
8	20070101	250	TMAX
9	20070101	256	TMAX
10	20070101	278	TMAX

The above Image shows the data preview of tmax table which is generated from the above code, and we can see that data scanned was 40.80MB and run time is 1.97seconds.

Data

Data source: AwsDataCatalog

Database: weatherdata

Tables and views:
 Tables (2): by_year, late20th
 Views (1): tmax

Query 4

```

1 SELECT date / 10000 AS Year, AVG(observation) / 10 AS Max
2 FROM tmax
3 GROUP BY date / 10000
4 ORDER BY date / 10000;
5

```

SQL Ln 5, Col 1

Run again **Explain** **Cancel** **Clear** **Create**

Query results **Query stats**

Completed Time in queue: 65 ms Run time: 28.982 sec Data scanned: 2.26 GB

Results (66) **Copy** **Download results**

This is the final query that we are performing, and we can see that I also did run successfully with 28.982 seconds of runtime and 2.26GB of data.

Completed Time in queue: 65 ms Run time: 28.982 sec Data scanned: 2.26 GB

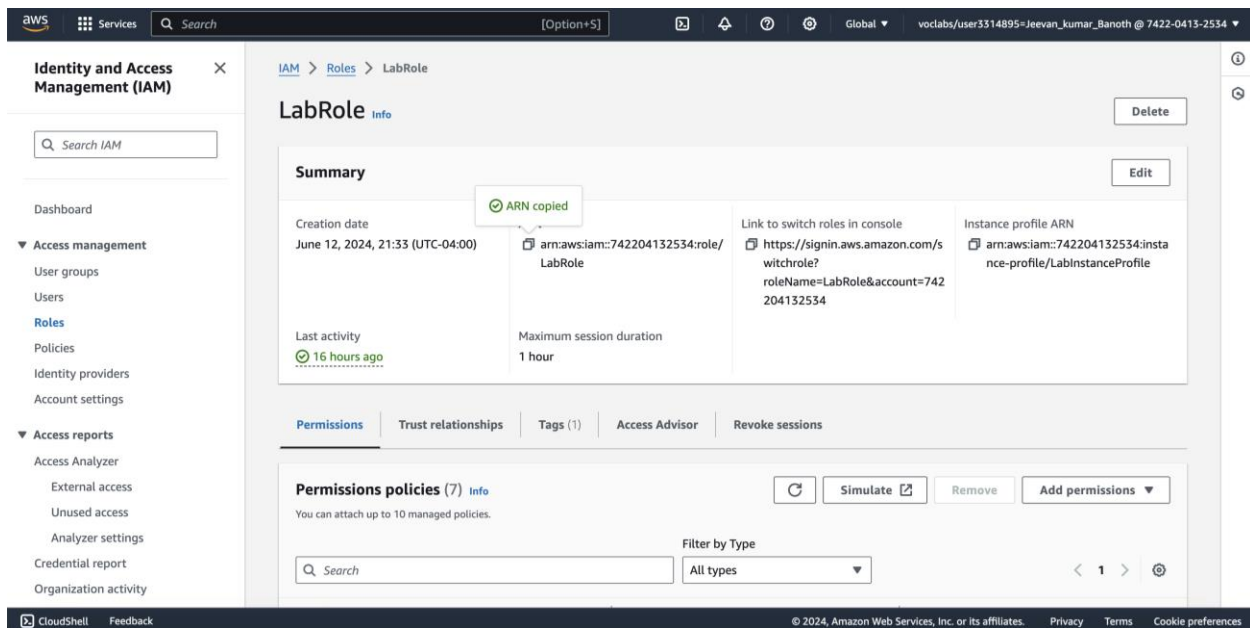
Results (66) **Copy** **Download results**

Search rows

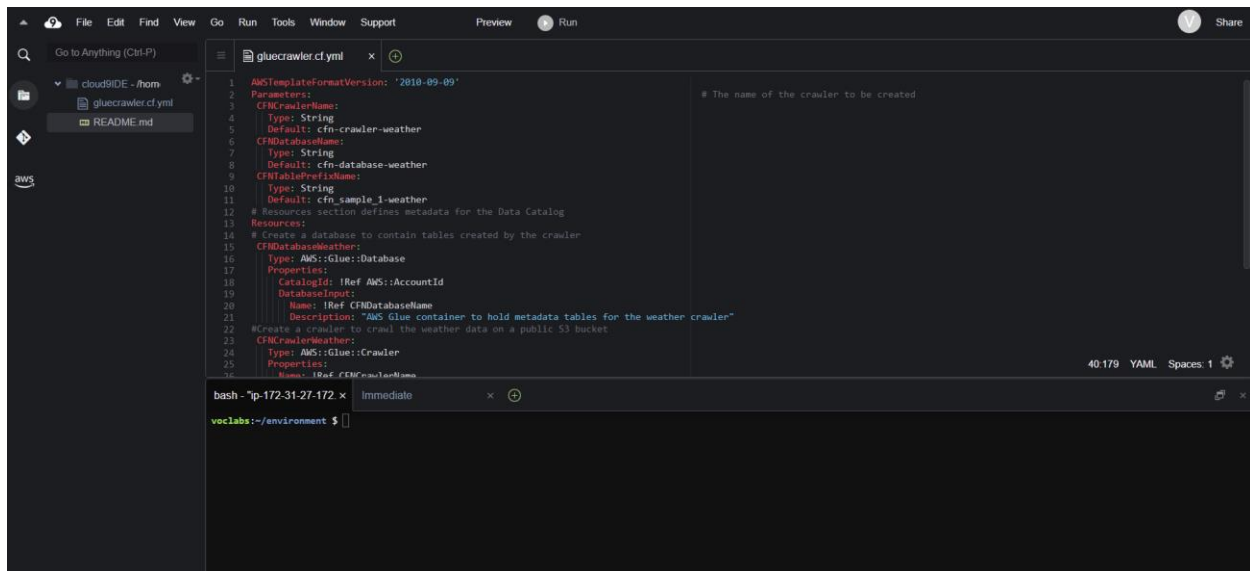
#	Year	Max
1	1950	16.743529427566383
2	1951	16.808930561471833
3	1952	17.283431803399885
4	1953	17.921563821822918
5	1954	17.481620577242513
6	1955	17.107016074678217
7	1956	17.157362940491474
8	1957	17.40718743879089
9	1958	17.303095718505677
10	1959	17.380660839916082
11	1960	16.933512950708725

Above is the output produced from the query and which is 66 columns and has Year and max values in it.

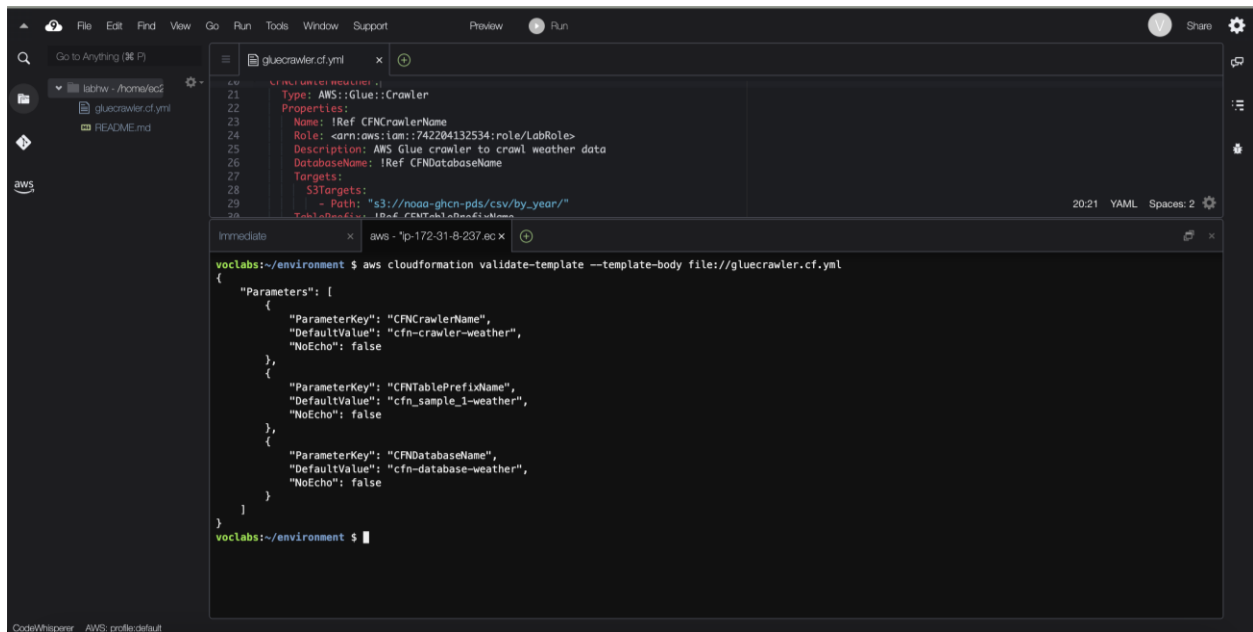
3. Create a CloudFormation template for an AWS Glue crawler:



In this task, we need to go to the IAM and then roles in that the Role that we gave during the crawler creation, so I gave it as LabRole then here we need to copy the ARN, as you can see above, I did copy that.



As shown above, we need to save that file by making some changes, which is to replace the ARN which we have copied earlier into here and save the file with the given name and extensions.

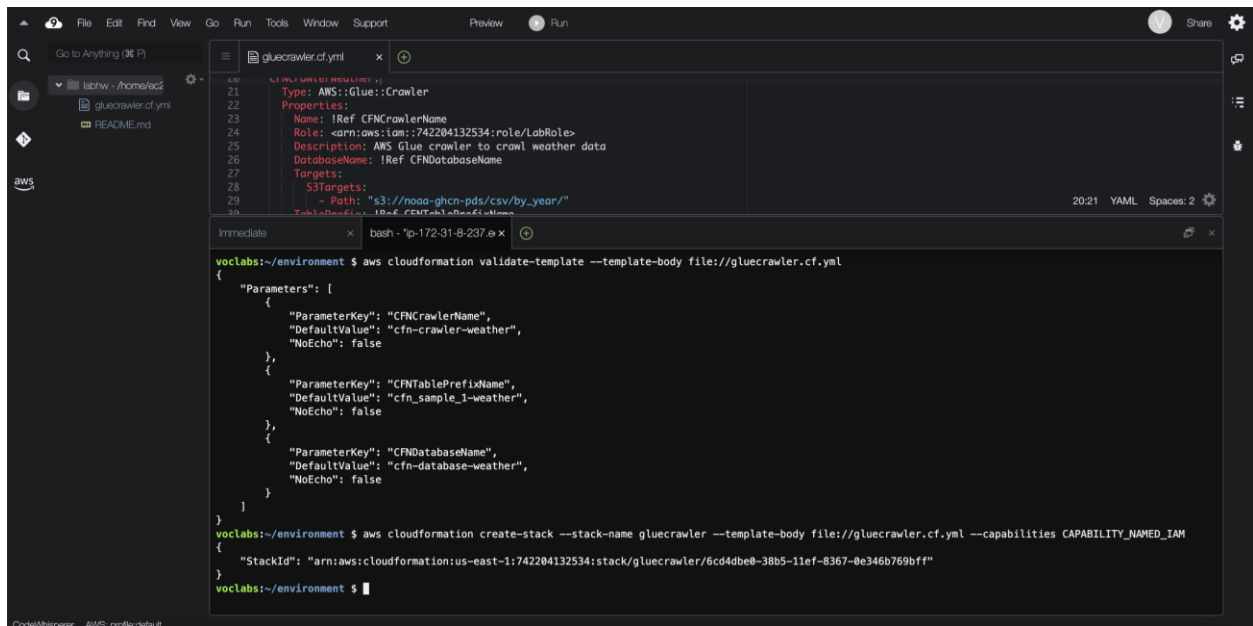


The screenshot shows a code editor with a file named `gluecrawler.cf.yml` open. The file contains a CloudFormation template for an AWS Glue crawler. The terminal window shows the command `aws cloudformation validate-template --template-body file://gluecrawler.cf.yml` being executed, which returns a JSON response indicating the template is valid.

```
21 Type: AWS::Glue::Crawler
22 Properties:
23   Name: !Ref CFNCrawlerName
24   Role: <arn:aws:iam::742204132534:role/LabRole>
25   Description: AWS Glue crawler to crawl weather data
26   DatabaseName: !Ref CFNDatabaseName
27   Targets:
28     - Path: "s3://noaa-ghcn-pds/csw/by_year/"
29     - Path: "s3://noaa-ghcn-pds/csw/by_year/"
30     - Path: "s3://noaa-ghcn-pds/csw/by_year/"
```

```
voclabs:~/environment $ aws cloudformation validate-template --template-body file://gluecrawler.cf.yml
{
  "Parameters": [
    {
      "ParameterKey": "CFNCrawlerName",
      "DefaultValue": "cfn-crawler-weather",
      "NoEcho": false
    },
    {
      "ParameterKey": "CFNTablePrefixName",
      "DefaultValue": "cfn_sample_1-weather",
      "NoEcho": false
    },
    {
      "ParameterKey": "CFNDatabaseName",
      "DefaultValue": "cfn-database-weather",
      "NoEcho": false
    }
  ]
}
```

Now in the terminal window we must use the codes that are given in the manual to check whether the file is correctly written or not.



The screenshot shows the same code editor with the `gluecrawler.cf.yml` file. The terminal window now shows the command `aws cloudformation create-stack --stack-name gluecrawler --template-body file://gluecrawler.cf.yml --capabilities CAPABILITY_NAMED_IAM` being executed, which returns a JSON response indicating the stack was created successfully.

```
voclabs:~/environment $ aws cloudformation create-stack --stack-name gluecrawler --template-body file://gluecrawler.cf.yml --capabilities CAPABILITY_NAMED_IAM
{
  "StackId": "arn:aws:cloudformation:us-east-1:742204132534:stack/gluecrawler/6cd4be0-38b5-11ef-8367-0e346b769bff"
}
```

See here, we are checking it by using stack numbers and Id.

```
Immediate x aws - "ip-172-31-8-237.ec x
voclabs:~/environment $ aws cloudformation create-stack --stack-name gluecrawler --template-body file://gluecrawler.cf.yml --capabilities CAPABILITY_NAMED_IAM
{
  "StackId": "arn:aws:cloudformation:us-east-1:742204132534:stack/gluecrawler/6cd4dbe0-38b5-11ef-8367-0e346b769bff"
}
voclabs:~/environment $ aws glue get-databases
{
  "DatabaseList": [
    {
      "Name": "weatherdata",
      "CreateTime": "2024-07-02T03:44:58+00:00",
      "CreateTableDefaultPermissions": [
        {
          "Principal": {
            "DataLakePrincipalIdentifier": "IAM_ALLOWED_PRINCIPALS"
          },
          "Permissions": [
            "ALL"
          ]
        }
      ],
      "CatalogId": "742204132534"
    }
  ]
}
voclabs:~/environment $
```

```
Immediate x aws - "ip-172-31-8-237.ec x
{
  "DatabaseList": [
    {
      "Name": "weatherdata",
      "CreateTime": "2024-07-02T03:44:58+00:00",
      "CreateTableDefaultPermissions": [
        {
          "Principal": {
            "DataLakePrincipalIdentifier": "IAM_ALLOWED_PRINCIPALS"
          },
          "Permissions": [
            "ALL"
          ]
        }
      ],
      "CatalogId": "742204132534"
    }
  ]
}
voclabs:~/environment $ aws glue list-crawlers
{
  "CrawlerNames": [
    "weather"
  ]
}
voclabs:~/environment $
```

```
File Edit Find View Go Run Tools Window Support Preview Run
Go to Anything (36 P)
labriv - /home/labriv
cloudformation delete-st
d
gluecrawler.cf.yml
README.md
aws
gluecrawler.cf.yml x
aws - "ip-172-31-8-237.ec x
Immediate (Javascript) (brow x) aws - "ip-172-31-8-237.ec x
{
  "Crawler": {
    "Name": "cfn-crawler-weather",
    "Role": "LabRole",
    "Targets": {
      "S3Targets": [
        {
          "Path": "s3://noaa-ghcn-pds/csv/by_year/",
          "Exclusions": []
        }
      ],
      "JdbcTargets": [],
      "MongoDBTargets": [],
      "DynamoDBTargets": [],
      "CatalogTargets": [],
      "DeltaTargets": [],
      "IcebergTargets": [],
      "HudiTargets": []
    },
    "DatabaseName": "cfn-database-weather",
    "Description": "AWS Glue crawler to crawl weather data",
    "Classifiers": [],
    "RecrawlPolicy": {
      "RecrawlBehavior": "CRAWL_EVERYTHING"
    },
    "SchemaChangePolicy": {
      "UpdateBehavior": "UPDATE_IN_DATABASE",
      "DeleteBehavior": "LOG"
    },
    "LineageConfiguration": {
      "CrawlerLineageSettings": "DISABLE"
    },
    "State": "READY",
  }
}
```

After performing all the commands in the end, we must check if the file is correctly updated or not which we can know that by seeing at the output in the end and when it says READY that means the file has been updated successfully. Else we need to check the code in the file.

Conclusion:

Analysis of AWS Glue and CloudFormation: A managed extract, transform, and load (ETL) service is AWS Glue.

CloudFormation is an AWS service which enables defining of infrastructure as code through templates.

Setting up a Database Schema and Glue Crawler Configuration: In creating of tables used a database schema for employee, department, project, and dependencies.

An AWS Glue crawler was configured that scours for data on an S3 bucket like weather in CSV format.

I created the Glue Database and Glue Crawler resources using CloudFormation.

Here is an overview of the CloudFormation Template: Example an AWS CloudFormation YAML template snippet that defines resources like AWS Glue Database and Crawler.

This template snippet contains parameters for customizable resource names such as database name, crawler name, and table prefix.

Discussed the importance of secure access control and permissions management for AWS services.

Successfully set up and configured an AWS Glue crawler using CloudFormation, enabling automated data ingestion from S3 into a Glue Database.

Demonstrated the practical application of AWS services for ETL tasks and access management in a controlled environment.